// BENCHMARK · METHODS · REVERSE ENGINEERING · LLM AGENTS

RETROGRADE: A Procedurally Generated, Behaviorally Graded, Provenance-Aware Benchmark for Reverse-Engineering Agents

vmfunc

FLOCK CAPITAL

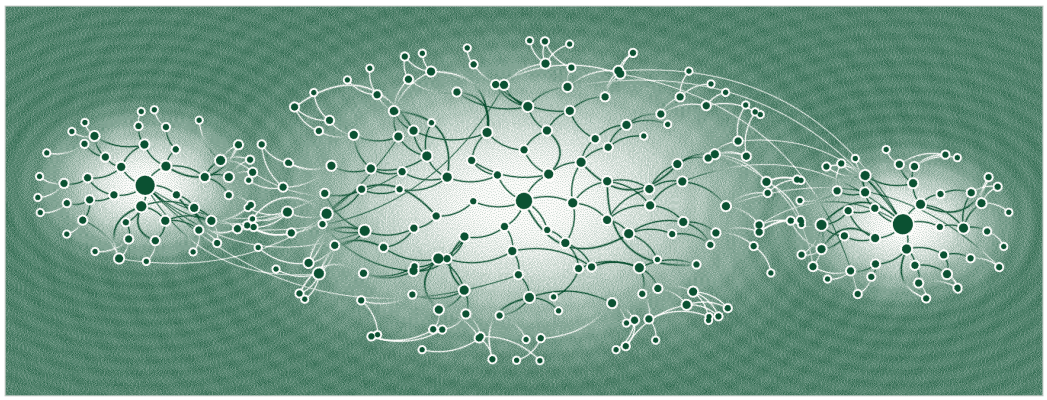
Lamb Wakefield

FLOCK CAPITAL

Laura K. (Alula)

FLOCK CAPITAL

November 1, 2026



CONTROL-FLOW · SEED 0XC0DE · FLOYD–STEINBERG ERROR DIFFUSION

ABSTRACT

Reverse-engineering (RE) benchmarks for LLM agents today are static, hand-built, expensive, and contaminating: a fixed corpus of targets, once published, leaks onto leaderboards and into training data, and is scored by flag-match or text similarity that rewards pattern-matching over understanding. RETROGRADE replaces the corpus with a generator*. Targets are synthesized from a compositional behavior grammar, each unique, each shipped with a machine-readable ground-truth specification emitted* by construction *rather than recovered after the fact. A claim about a binary is graded by* executing*—differential testing against a reference oracle, output prediction on adversarial inputs, and causal interventions verified in an emulator—never by string similarity. Every claim carries a confidence and a provenance pointer, scored by a proper scoring rule with asymmetric loss so that a confident wrong claim about a safety-relevant fact costs more than an honest "I don't know." Because targets are generated, the benchmark runs as a self-renewing live leaderboard: fresh unseen seeds each round, IRT-estimated item difficulty, saturation retirement, and a canary protocol embedded in every artifact. We implement a generator with fifteen behavioral archetypes (protocol, bytecode VM, data transform, container format, benign malware-behavior surrogate, license-key validator, fixed-point sensor and DSP signal paths, a save-file loader, a config parser, a template renderer, and four systems workloads: an X.25 link layer, an automotive ECU, a motor controller and a command interpreter), each compiled natively to aarch64 Linux or re-hosted on four bare-metal ISAs (Cortex-M, RV32, PowerPC e500, MSP430) or the Hexagon DSP, optional seeded per-instance platform settings (memory map, string representation, number format), cross-language code generation (C, C++, Rust, Go), a composable ten-pass protection lattice, a deterministic tamper-resistant grader with a reward-hacking audit as a CI gate, and a scaffold-agnostic agent harness that drives frontier models through OpenRouter, each served only by its maker's first-party API.* ***Methods paper.*** *This preprint describes the design, the verified implementation, and the frontier-evaluation protocol: the pre-registered full evaluation (v1: eight archetypes, three protection tiers, five seeds, two samples; pending funding), and a smaller* Round 0 *preview of 13 frontier models on the same eight archetypes and three tiers with one seed and one sample per cell, run blind under a published commit–reveal seed commitment, with results reported as capability under protection pooled across models and as score bands, not as a ranking. The paper reports no model results; they are published separately.*

1 Introduction

A reverse-engineering benchmark has a short half-life: the moment it is published, its binaries leak onto leaderboards and into the next pretraining run, and a flag-match score rewards an agent for *recognizing* a target it has seen rather than *understanding* one it has not. Retrograde is built the other way around. There is no corpus to leak—only a generator that mints a fresh, never-before-compiled program on demand, hands the grader a machine-readable account of exactly what that program does, and asks the agent not "what is the flag" but "what does this binary *do*, how do you know, and how sure are you." Answers are checked by running them.

The metric lineage of reverse-engineering (RE) evaluation runs token-match → recompilability → re-executability → recompilable-and-integrable, with the frontier being *behavioral equivalence under test* [3]. A documented "compile-correct gap" of 57–68 percentage points—between code that compiles and code that is semantically correct—shows that text- and compile-level metrics are insufficient. On the agentic side, Retrograde-Static's

closest predecessor, SRE-Bench [5], hand-built 19 private programs (~16.9K LoC each), 262 binary instances and 1,572 deterministically graded tasks at a cost of 5,000+ expert hours. It is the strongest prior design, and it is *static*: once on a public leaderboard it will contaminate and saturate, and it can ablate anti-analysis protections only coarsely. BinMetric [1] and CTF suites such as Cybench [2] are contaminated by construction (public OSS and public CTF challenges).

Retrograde's thesis is that *generation is the structural fix*. If targets are synthesized on demand with ground truth emitted *by the generator*, then contamination is impossible by construction, grading is deterministic and behavioral, difficulty auto-calibrates, and the benchmark renews itself. We additionally measure axes no existing RE benchmark measures: causal and counterfactual understanding of a binary, the provenance of each claim, and calibrated uncertainty.

Contributions. (1) A procedural generator that emits novel, real-world-plausible programs with a machine-readable behavioral specification (§3); (2) a behavioral, causal, tamper-resistant grader with provenance and calibration scoring and a reward-hacking CI gate (§4); (3) a self-renewing live-leaderboard controller (§5); and (4) a scaffold-agnostic agent harness that drives frontier models under one frozen configuration, with first-party routing, blind instances, commit–reveal seeds and per-cell provenance (§6.1). We state five novelty claims and the ablations that establish each (§6). The full pre-registered frontier evaluation (v1) is pending funding; a smaller, blind Round 0 preview of 13 models uses the same protocol (§6.2), with results framed as capability under protection and as score bands rather than a ranking. This paper describes the approach only; model results are published separately.

2 Design: the spine

Two artifacts fix everything downstream: the behavior IR and the ground-truth contract. The governing rule is that every gradeable fact is *emitted by the generator from the IR*, never recovered by analyzing the compiled binary.

Invariants. **Single source of truth**: one IR object per instance, from which the executable `oracle`, the target source, the specification, and the provenance map are all derived by pure functions. **Determinism**: `build(seed, archetype, params)` is pure given a pinned toolchain fingerprint. **Self-consistency gate**: before an instance ships, the `oracle` and the compiled target must agree on fuzzed and boundary inputs; a disagreement is a generator bug and the instance is rejected. **Solvability gate**: every trigger task must have a constructible firing input whose execution reaches the guarded block, checked by IR tracing. **Ground truth lives outside the agent sandbox. Behavioral grading only**: claims are graded by executing or by exact structured match, never by string similarity.

Behavior IR. A typed, serializable tree of value/expression/ statement nodes, functions, and a program entry over a global input buffer. All arithmetic is `uint32` with wraparound so the Python interpreter (the `oracle`) and every compiled backend agree bit-for-bit. The IR also has constant tables and a few `int32` views of those words (signed comparisons, arithmetic shift), on which the backends agree the same way. Archetypes are parameterized

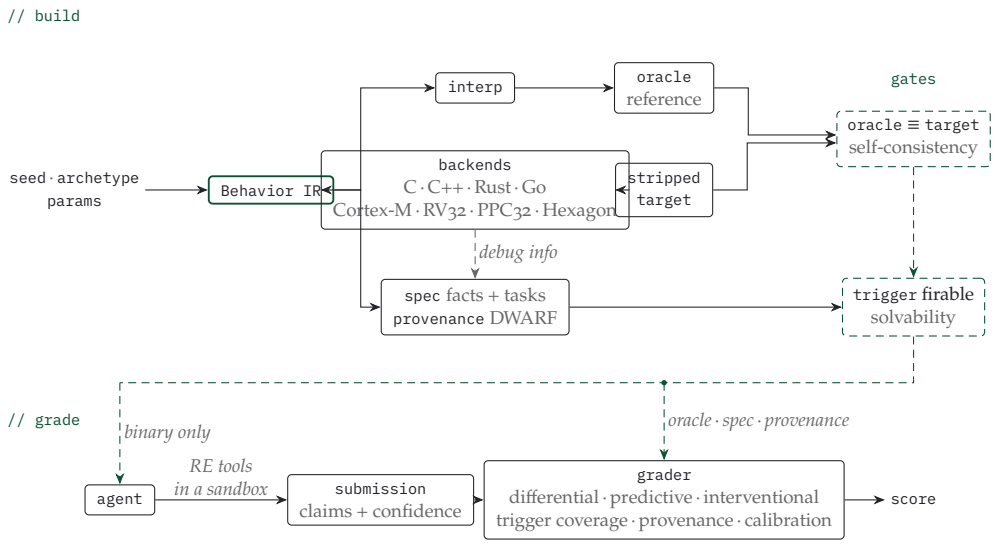


FIGURE 1 · The RETROGRADE pipeline. One behavior IR yields a reference oracle (an interpreter), a stripped target (via C/C++/Rust/Go, Cortex-M, RV32, PowerPC, MSP430 and Hexagon backends), and a by-construction specification with DWARF provenance. Two gates release the bundle: oracle ≡ target (self-consistency) and trigger-firable (solvability). The agent receives the binary and the task prompts; the grader holds the oracle, specification and provenance, and scores by differential, predictive, interventional, coverage-verified trigger, and provenance checks under a calibration rule.

functions `params` → IR that sample structure (field layouts, opcode tables, state machines, transforms) from the seed within ranges chosen to look like real firmware/parsers/VMs.

Ground-truth contract. The specification is a versioned JSON document of `Facts`—the atom of grading—each carrying a canonical value, a `kind`, a provenance anchor (resolved to address ranges after compilation via DWARF), a safety weight, and a verifier family. `Tasks` reference facts and name the exact I/O the agent must produce. The agent's answer schema requires, per claim, a value, a confidence in $[0, 1]$, and a provenance pointer.

What the agent is told. By default the agent-visible manifest names the archetype, seed, target kind, language and applied protections. A *blind* instance replaces all of it with an opaque instance id, an HMAC of the build parameters under a private key, so it cannot be inverted to the seed. A re-hosted target then ships under the neutral name `target` (a bare-metal one as a raw image with no container header), and on the Cortex-M, RV32 and PowerPC kinds identifying its instruction set (from a 19-entry vocabulary) becomes a recovery fact. Blind mode hides only the metadata the agent would otherwise be handed: the binary keeps its toolchain traces (the compiler's `.comment` section, runtime libraries), because recognizing the toolchain from evidence is part of the task. The full manifest stays with the grader.

3 Generator

Archetypes. Fifteen behavioral (logic) archetypes ship: `protocol` (binary wire protocol with framing, CRC and a guarded state machine), `vm` (register bytecode interpreter with a per-seed randomized opcode table), `transform` (mixed-boolean-arithmetic byte transform), `format` (container with a record directory and a guarded handler), `malware_surrogate` (a benign C2/persistence simulacrum with a hidden multi-step arming sequence; §9), `keycheck` (a license-key validator: a seeded chain of checks over disjoint regions of the key—a magic tag, an affine relation between two bytes, an additive checksum and a masked serial comparison—whose headline task is producing a key it accepts), `temp_sensor` (a thermostat signal path: a piecewise-linear calibration table, an exponential moving-average filter and a latching over-temp cutout) and `dsp` (a fixed-point audio filter: a short FIR stage and a biquad section with a saturating clamp and a clip detector), `savefile` (a versioned save loader: a version byte selects the record layout, guarded by an additive checksum), `config` (a typed TLV configuration parser with clamped limits and an enable key) and `template` (a byte-stream template renderer with sigil directives over a variable table); four systems workloads — `x25` (an HDLC/LAPB link layer: flag framing, byte-stuffing, a CRC16 FCS and a modulo-8 windowed I/S/U state machine), `ecu` (an automotive ECU: CAN framing, UDS-over-ISO-TP reassembly, session-gated read-by-identifier, and a seeded 2D calibration map with integer interpolation), `scooter` (a BLDC motor controller: six-step hall commutation, a PI limiter with anti-windup, and a latched fault path) and `interp` (a small expression/command interpreter with a seeded opcode table). Each compiles natively to an aarch64 Linux ELF and can be re-hosted on any target kind below as `<logic>@<kind>`; the registry also names `firmware` and `firmware_riscv` (the protocol workload as a bare-metal Cortex-M or RV32 image). Each guards a deep branch whose trigger the agent must reach. A `size` knob scales programs (`huge` ≈ 20 states, 150+ transitions, ~3.8K instructions bare and ~22K under the full protection stack for the protocol tier).

Cross-language and cross-ISA backends. One IR lowers to **C, C++, Rust, and Go**—each byte-identical to the `oracle`—so "which language is easiest to reverse" is a measurable axis. The firmware tier re-hosts the same IR on **four bare-metal ISAs**: ARM Cortex-M (Thumb), RISC-V RV32, big-endian 32-bit PowerPC (e500) and 16-bit TI MSP430 (run under an in-repo Python core emulator, since no QEMU/Unicorn models it), each with modeled MMIO, and additionally runs as a static Linux user-mode binary for the Hexagon VLIW DSP (v73, raw syscalls, no libc) under `qemu-hexagon`, whose instruction packets, duplexes and constant extenders come from the compiler's packetizer. Any logic archetype runs on any of them (e.g. `keycheck@fw_ppc`), so "which ISA is easiest to reverse" is measurable on the same IR program. The RV32 and PowerPC compilers are clang configuration files kept in the repository (RV32 uses gcc instead when one is installed; Cortex-M uses `arm-none-eabi-gcc`), so those targets need only an LLVM install; a CI step builds a probe with every configuration and checks what it produces.

Mixed-compiler targets. A native instance can be split into two translation units built by different C compilers (gcc and clang) and linked with one driver; the per-TU compiler, the linked C runtime (compiler-rt vs libgcc) and whether the `.comment` section is kept are seeded. The result is one binary carrying both compilers' code and version strings, so toolchain traces are mixed or outright misleading, as in real multi-compiler builds. Recovering which compiler produced each function is a recovery task—one fact per function, keyed

to its address range, graded against the build map and regenerated by the grader from the seed. It is paired with a verified catalogue of source idioms the two compilers lower differently (`generator/idioms.py`, `docs/compiler-idioms.md`): 17 shapes at the native tier's `-O1`, each compiled in the sandbox with gcc 13 and clang 18, disassembled, and asserted to differ in the documented way, with both binaries checked against a Python reference so the helpers are output-invariant. Examples: clang folds a SWAR bit reversal to `rbit` and a shift-mask byte swap to `rev` while gcc keeps the arithmetic; gcc recognises the SWAR popcount (`cnt`/`addv`) and the De Bruijn count-trailing-zeros table (`rbit`/`clz`) at `-O1` while clang does not; clang turns an if-chain over dense keys into a value table or a linear map and a chain of constant-argument calls into one argument table plus a tail call, where gcc emits compare chains. Two of these invert the direction seen on an x86-64 host with newer compilers, which is why the catalogue is verified on the toolchain that builds the targets rather than taken from folklore. The catalogue is a seeded, default-off knob in the same shape as the string dialect, and it is wired through the emitter: when enabled, the selected helpers are compiled into the target (called into a `volatile` sink, so they survive dead-code elimination without touching stdout—the `oracle` is unchanged and self-consistency still holds). On the mixed-compiler target the selected ids are split across the two toolchains by the seed, so each helper becomes a decisively compiler-specific entry in the per-function compiler-identification task, with address-range provenance, graded tamper-proof by regenerating the split from the seed.

Per-instance platform settings. An instance can also draw its platform from the seed. Each setting is an independent knob, off by default (except the memory map on PowerPC), so it can be ablated on one fixed program the way a protection is. **Memory map**: the flash, RAM and peripheral base addresses, the serial-port register offsets, the order functions are placed in, where `.rodata` goes and a padding gap are all seeded; a reset stub keeps the entry point at the flash base while the code moves. The load base and register addresses are recovery facts. **String representation**: event labels are stored NUL-terminated, length-prefixed, as a pointer-and-length descriptor without a terminator, or as NUL-terminated 16-bit code units that are narrowed when printed. The printed bytes do not depend on the choice, so the `oracle` is unchanged and the output does not reveal the representation; it is a recovery fact, found by reading the stored labels and the print routine. **Number format**: emitted values are printed in a seeded base, case, prefix and zero-pad width. This setting changes the output; the `oracle` and all four language backends render numbers with the same algorithm, and when it is enabled, output-prediction tasks cannot assume decimal. **Stdlib API shape**: the program's internal helper surface (the bounded accessor, the output path) is routed through a seeded "invented libc" with a seeded wrapper name, argument convention ((ptr,len) vs (len,ptr) vs a {ptr,len} descriptor) and error convention (negative-return vs out-param errno vs sentinel). It is output-invariant, so the `oracle` is unchanged and the shape is a pure static-recovery fact. The IR also has constant tables (8-, 16- or 32-bit entries behind a bounds-checked accessor) and signed 32-bit comparisons and arithmetic shift; the `oracle` and every backend agree on them.

Protection lattice. Ten composable, behavior-preserving passes apply to the same seed: `mba`, opaque predicates, optimizer-robust constant hiding (`opaque_const`), control-flow flattening, string/label encryption, anti-debug, an optimization sweep, virtualization-obfuscation (an Expr→bytecode VM), packing (in-place `.text` self-decryption), and

compiler-customized internal calling conventions (`ipa_cc`: seeded arithmetic subtrees are outlined into `noinline` helpers, which the compiler then compiles off the platform ABI with interprocedural register allocation (clang) or IPA cloning (gcc), plus a seeded explicit convention per helper on native targets). Because the `oracle` is built from the unprotected IR and the self-consistency gate re-checks equivalence, any protection that breaks behavior is rejected automatically—so obfuscation can be aggressive and remains safe.

Encrypted firmware. An opt-in `fw_crypt` knob ships a bare-metal image whose payload is encrypted (optionally compressed) with a seeded real cipher —AES-128/256 (ECB/CBC/CTR), RC4 or ChaCha20, our implementations checked against published test vectors—behind a boot stage that derives the key from material in the image, decrypts into RAM, runs a seeded integrity check (SHA-256/SHA-1/CRC-32/ HMAC) and jumps to the payload. It is decryptable from the shipped image alone, and the self-consistency gate proves it: the `oracle` equals the *emulated, self-decrypting* image. The cipher, mode, key, IV/nonce, encrypted ranges and integrity scheme are recovery facts, regenerated by the grader from the seed.

Bank-switched firmware. An opt-in `bank` knob (RV32, Cortex-M Thumb and big-endian PowerPC) reaches the program's constant tables not directly but through a *bank-switched data window*, as memory-constrained firmware does: the tables are packed into a seeded number of equal-size banks in a physical backing region, and a fixed window in the address map aliases one bank at a time, chosen by a bank-select register. A boot stage fills the banks; each table access writes the select register and reads the window. The select can be *delayed*—taking effect only after a seeded number of further window accesses—which the generated firmware covers with that many dummy reads. It is output-invariant: a correct bank model reads back exactly the stored values, so the `oracle` is unchanged and the self-consistency gate (`oracle` equals the emulated, bank-reading image) proves the banking is modeled correctly. The window base, the bank-select register address, the bank count, the bank stride, the backing base and the select delay are recovery facts, regenerated by the grader from the seed. (Code overlays—banking the *instruction* stream—are future work; the window translation is modeled in the re-hosting emulator, where instruction fetch needs real backing memory.)

Custom MMU. A sibling `mmu` knob (same three ISAs: Cortex-M, RV32, PowerPC) reaches the tables through a *custom* memory-management unit rather than a flat select register: the firmware builds a page table in RAM at boot, and accesses to a virtual region are translated by walking it. The MMU is deliberately non-standard—the page-table entry's field layout (the frame-number shift and the present/writable/locked bit positions) is *seeded*, so no architectural prior transfers and the agent must recover a format, not a constant. Some pages are locked by a guard engaged after boot (a guarded region that faults if touched, but which the program never accesses on graded inputs, so behavior is unchanged). It is output-invariant for the same reason as the bank window—a correct walk reads back the stored values—and the self-consistency gate (`oracle` equals the emulated, page-walking image) proves the walk is modeled correctly. The virtual base, the page-table base, the backing base, the page size, the page count, the PTE frame shift and present/writable/locked bit positions, and the locked-page list are recovery facts, regenerated by the grader from the seed.

Generated platform. A `platform` knob (Cortex-M) ships the firmware in a *custom executable format* with an in-image loader, rather than as a raw flash image. The payload is relinked to run from a RAM execution region, split into several segments that are placed in a container in a seeded, shuffled order and XOR-scrambled, behind a seeded header and segment table. A loader boot stage at the reset vector parses the container at run time and reassembles the segments into the exec region—a bespoke relocation/segment encoding—before jumping to the payload. The payload is reconstructable from the shipped bytes alone, and the self-consistency gate proves it: the `oracle` equals the emulated image once the loader has run (no emulator support is needed, since the loader writes into ordinary RAM). The container magic, the segment count and entry offset, the exec base, the payload XOR key, the header offset, the container's byte range in the image and the segment table (each `[src,dst,len]`) are recovery facts, regenerated by the grader from the seed.

Auxiliary-core mailbox. A `mailbox` knob (the three Unicorn-hosted ISAs: Cortex-M, RV32, PowerPC) adds a fixed-function second core behind an MMIO mailbox: the firmware writes an argument and a command, the aux core computes a seeded transform (`rotl(arg,r)^k+c`), and after a seeded latency—a polling deadline—a status bit latches and the firmware reads the result. At boot the firmware handshakes with it (a seeded challenge, verified against a baked expected value, halting on mismatch), which is output-invariant on a correct model yet gate-exercised: a wrong aux-core model fails the handshake and diverges, so the self-consistency gate proves the coprocessor is modeled correctly. The mailbox base, the four register addresses, the transform constants and the latency are recovery facts. (This captures the modeled-coprocessor half of a richer board; cycle-scheduled asynchronous interrupts are intentionally out of scope, as nondeterministic timing cannot be reconciled with the deterministic `oracle`.)

Security coprocessor (custom ISA). A `seccop` knob (the same three Unicorn-hosted ISAs) adds a *programmable* coprocessor with a seeded custom micro-ISA: the firmware uploads a short bytecode program to a code window, writes an argument and triggers it, and the coprocessor runs a small accumulator machine (`acc <op>= imm8` per two-byte instruction, then `HALT`) whose opcode byte for each operation is per-instance. The generator is the in-repo assembler; the re-hosting emulator is the interpreter. At boot the firmware runs the uploaded program on a seeded challenge and halts on a wrong result, so it is output-invariant on a correct ISA model and gate-exercised on a wrong one. The coprocessor base and registers, the opcode table (operation → byte) and the uploaded program bytes are recovery facts; recovering the opcode table is itself a dynamic-analysis exercise (probing the coprocessor), since the instruction semantics live in the emulated device rather than in the image. This is the aligned, verifiable core of a custom-ISA re-hosting tier; a full third-party ISA such as Xtensa would be a much larger assembler/emulator effort and is left to future work.

4 Grading

A claim is graded by *executing*, never by comparing strings. Six verifier families run: **exact** structured match (field offsets, opcode tables); **differential** (the agent's reconstructed source is compiled and behaviorally compared to the `oracle` on fuzzed and boundary inputs); **predictive** (the agent predicts the binary's output on grader-chosen adversarial

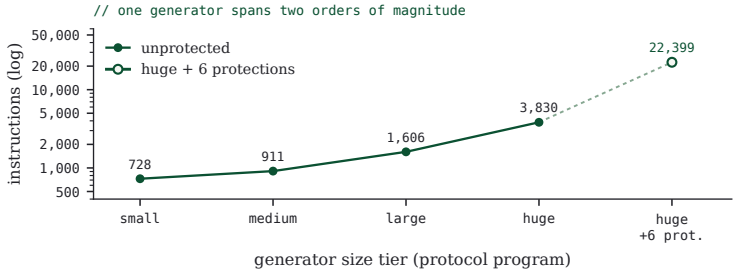


FIGURE 2 · Scaling. The `size` knob grows a protocol target from ~700 to ~3.8K instructions; the full protection stack lifts the `huge` tier to ~22K, most of which is obfuscation rather than program logic. SRE-Bench's ~16.9K per program counts lines of source, so the two scales are not directly comparable. Every instance remains deterministically, behaviorally gradeable.

inputs); **interventional/causal** ("if you patch byte X / force branch Z, what changes?"—verified against ground truth); **trigger** (the agent supplies an input that must actually reach a guarded block, verified by a debugger-free coverage rebuild immune to anti-debug); and **provenance** (a cited address must land in the fact's DWARF range).

Scoring. Correctness per fact is combined with a Brier proper scoring rule [4] over confidences, an *asymmetric loss* (a confident-wrong claim on a high-safety-weight fact costs more than abstention), and a *provenance discount* (correct-but-unsupported claims earn a fraction). Over the N graded claims of an episode, with correctness $c_i \in [0,1]$, confidence $p_i \in [0,1]$ and safety weight w_i, the composite is

$$S \;=\; \frac{1}{N}\sum_i \pi_i c_i \;-\; \frac{1}{2}\cdot\frac{1}{N}\sum_i (p_i - c_i)^2 \;-\; \frac{1}{N}\sum_{i:\, c_i<\frac{1}{2},\, p_i>\frac{1}{2}} 2\,(p_i-\tfrac{1}{2})\,w_i,$$

where $\pi_i = 0.4$ if the claim needs provenance and its cited address does not land in the fact's range, and 1 otherwise. $S \le 1$, and a confidently wrong agent scores below zero. Safety weights are set per fact by the generator (on `protocol`, e.g., 2.0 on the trigger condition and 1.5 on the CRC polynomial; 1.0 by default). The grader itself coerces confidences: anything non-numeric or non-finite counts as 0 and the rest is clamped to $[0,1]$; a malformed task entry is graded as an abstention, and only the first claim per fact is graded. The design principle: five correct findings do not cancel one confident, destructive wrong claim.

Tamper-resistance. The grader regenerates fact values and expected outputs from the seed; it never trusts any file the agent could touch. Editing the on-disk specification has no effect on scoring.

Reward-hacking audit (CI gate). Adversarial agents attempt to cheat each verifier; all must fail. Provenance forgery (correct values, bogus addresses) is discounted; a trigger output-spoof (printing the right token via a different path) is rejected because the verifier checks that the guarded block executed; reconstruction overfit fails randomized differential testing; ground-truth tampering has no effect. Four further gates attack the grader's input handling: malformed confidences (NaN, $\pm\infty$, strings), garbage submission schemas,

duplicated claims and claims on fact ids that were not asked must not raise the score. The eight gates run in CI.

5 Self-renewing live leaderboard

Because targets are generated, Retrograde runs as a rolling evaluation. Each round draws *fresh unseen seeds* from a 2^{62} seed space as a pure function of a private, high-entropy round key; the live controller records every draw in a persistent ledger so that no seed is reused across rounds, and a round key maps to a fixed, reproducible draw. Rounds use *commit–reveal*: before a round runs we publish a SHA-256 commitment to the round key, and after it closes we reveal the key. Anyone can check the revealed key against the commitment, so the seeds were fixed before the run and nobody, including us, could choose them after seeing results. Per-item difficulty is estimated with a one-parameter (Rasch) IRT model [8] fit to a reference-agent panel's response matrix; saturated items (solved by the whole panel) are retired. A canary string is embedded in every public artifact and baked into every target binary, so leakage of an instance or its solution into a training corpus is detectable by a single regex. The canary, like the blind instance id, is an HMAC under a private key: an unkeyed hash of (archetype, seed) would let an agent that reads the canary brute-force the seed, and official runs refuse to start without the key. A frozen Retrograde-Static slice for reproducibility (binaries and public tasks only; its release is pending) sits alongside the rotating Retrograde-Live slice—the structural answer to saturation that static sets cannot offer.

6 Evaluation

We state five novelty claims and the ablations establishing each. Design ablations (A1–A6) are run and reported here. The frontier-model evaluation has two levels under one frozen protocol (§6.1): the full pre-registered v1 design, pending funding, and a smaller Round 0 preview (§6.2). Model results are not part of this paper.

Claim 1 — contamination-proof by construction. Generating 150 fresh seeds per archetype yields 150/150 distinct programs for protocol, vm, format and malware_surrogate, and 145/150 for transform (whose constant space is smaller; collisions are the expected birthday rate). A conservative, code-computed lower bound on the protocol space is 5.9×10^{32}—over 10^9 by 23 orders of magnitude.

Claim 2 — behavioral, not textual. A reconstruction textually near-identical to the true source (SequenceMatcher ratio 0.9999) but behaviorally wrong by one flipped comparator scores 0.048 on differential grading. A text-similarity grader would overscore it by ~0.95.

Claim 3 — provenance and calibration. Two guessers with identical accuracy (0.062) score −0.76 (overconfident) vs. −0.05 (calibrated)— confident-wrong is punished ~14× harder than honest uncertainty (Figure 4). Correct answers with forged provenance earn 0.043 vs. 0.108 with honest provenance.

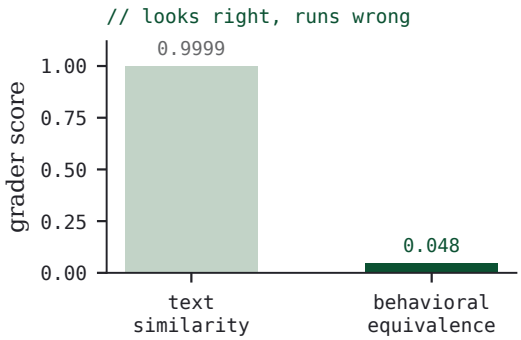


FIGURE 3 · Behavioral vs. textual grading. A reconstruction that is textually near-identical to the true source (similarity 0.9999) is behaviorally wrong (0.048 equivalence under differential test). Text-match grading would overscore it by ~0.95; RETROGRADE grades what the code *does*.

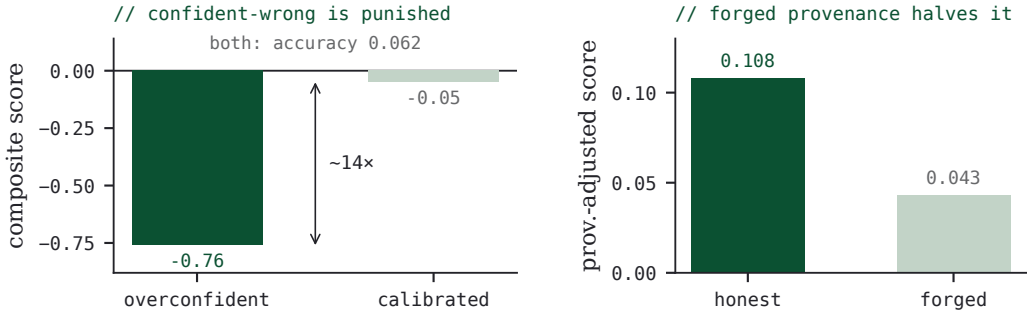


FIGURE 4 · Calibration and provenance scoring. *Left*: two agents with the same accuracy (0.062) are separated by an order of magnitude once a proper scoring rule and asymmetric loss price their confidence—bluffing does not pay. *Right*: forging the justification for a correct answer roughly halves its credit.

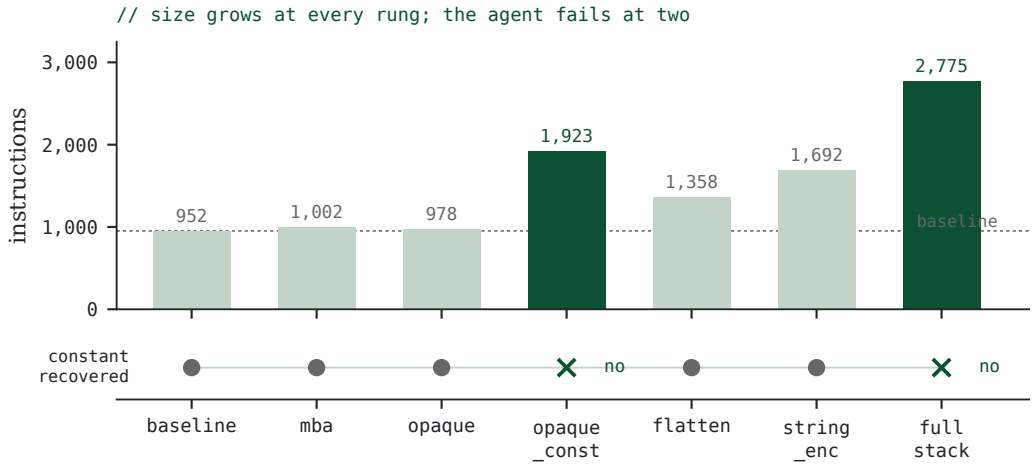


FIGURE 5 · The protection lattice on one fixed program. Disassembled instruction count climbs at every rung (top); a disassembly-first agent recovers the magic constant until `opaque_const` removes the lexical anchor, then fails (bottom). Same program, protection varied—the clean capability-drop ablation a static corpus cannot do.

TABLE 1 · Protection lattice on one seed (protocol, size=large). The magic immediate—a lexical anchor—survives most passes but is removed by `opaque_const`, and the static agent's recovery drops accordingly.

config	instrs	magic immediate	static agent recovers
baseline	952	present	yes
mba	1002	present	yes
opaque	978	present	yes
opaque_const	1923	**absent**	**no**
flatten	1358	present	yes
string_enc	1692	present	yes
full stack	2775	**absent**	**no**

Claim 4 — composable protection lattice with a capability-drop curve. For one fixed program, as protections stack the disassembled instruction count rises (baseline 952 to 2,775 under the full stack) and a disassembly-first agent's recovery of the magic constant drops from present to *absent* under `opaque_const` (Table 1).

Claim 5 — self-renewing. Fresh-seed sampling, IRT difficulty, and saturation retirement let the live slice rotate while a frozen slice stays fixed (§5); the reference panel's IRT fit ranks structure-recovery, trigger-discovery and reconstruction as the hardest tasks and estimates agent abilities (dynamic-analysis ≫ guesser).

Cross-language and self-consistency. Across the full matrix (C++, Rust and Go × five archetypes × seeds 0–9 × {small, large}, 300 builds), every target passes the self-consistency gate and grades 0.999 for the ground-truth oracle agent. Virtualization (50/50) and packing (10/10, section confirmed garbled) likewise pass.

Platform settings (A6). Across the three Unicorn-hosted ISAs (Cortex-M, RV32, PowerPC), four combinations of the per-instance settings (seeded memory map alone, and with the string representation, the number format, or both) and five seeds, all 60 builds pass the self-consistency gate and the ground-truth oracle agent grades at least 0.9986. The string representation and number format also pass in all four source languages (24 `keycheck` builds). Each of the five seeds draws a different memory map on every ISA. On `keycheck` with every setting off (five seeds) the reference panel separates: oracle 0.999, dynamic analysis 0.624, overconfident guesser −0.927. The dynamic-analysis agent answers the prediction and intervention tasks by running the validator and abstains on recovery, keygen and reconstruction.

6.1 Frontier evaluation protocol

The non-LLM reference panel (oracle upper bound, dynamic-analysis, static- analysis, overconfident guesser) anchors difficulty. Frontier models are driven through OpenRouter by one scaffold under one frozen configuration file; changing any field after a run starts means a new release-candidate tag and a fresh run.

Harness. The agent has three tools: `run` (a bash command in the sandbox, output capped at ~8 KB, 60 s default and 300 s maximum per command), `read_file` (an agent-visible instance file in windows of at most 8 KB) and `submit` (once; it ends the episode). The sandbox offers objdump, llvm-objdump, radare2, gdb, Python, clang and the usual binutils. Only the agent-visible files are staged into it, read-only at `/work`, so the ground truth is physically absent rather than merely forbidden. Containers run as `linux/arm64` with `–network none`, `–cap-drop ALL`, `no-new-privileges`, 1 GB of memory, one CPU and 256 processes. The sandbox image's base is pinned to a content digest, the packages that determine target bytes (clang 18, gcc 13, binutils, libc) and every Python dependency are version-pinned, and each instance and each cell records the image digest.

Budgets. Identical for every model: 30 tool-calling rounds, 90 tool calls in total (parallel calls count individually), 1,800 s of wall clock per episode, 16,384 completion tokens per call, and reasoning effort set explicitly to `high` rather than left to each provider's default. Temperature is the one sampling parameter left to the provider: it is omitted from every request for every model, because the first-party Anthropic and OpenAI reasoning endpoints reject it and Moonshot accepts only 1, so no single value can be sent to every endpoint. Every model receives the same notice when six and when three rounds remain, and when a budget runs out the harness forces a final `submit`. The forced call first names the `submit` tool; if the provider rejects a named tool choice (Anthropic, DeepSeek, Qwen, GLM and Muse do with reasoning enabled), the harness retries at once with `required` and then `auto` (Kimi needs `required`), so every model still reaches a final submission. The prompt is kept under 70% of the model's context window by eliding the oldest tool outputs (the six most recent stay verbatim): near the limit some providers cap the completion, which silently truncated forced submissions in an internal pilot.

Submission robustness. The harness parses near-JSON leniently (hexadecimal literals, comments, trailing commas, Python literals), maps a submission key that names a task *type* onto the one missing task of that type when this is unambiguous, and, if a `submit` call arrives with empty arguments, asks once for the same submission as plain text. A

submission missing tasks is bounced back once with the schema error. These rules apply identically to every model and are logged per cell. A cell whose episode made no API call, or in which an API call still failed after retries with exponential backoff, is recorded as an error and never as a score; resuming the run re-runs it.

Routing and caching. Every model is served *only* by its maker's first-party API: each model is pinned to that provider, fallbacks to other hosts are disabled, and only endpoints that honour every request parameter (tools, reasoning) are eligible. The reference deployment is the cleanest comparison; it removes third-party quantization as a confound and avoids host-specific tool bugs (one third-party host forwarded empty tool arguments for one model in an internal pilot). All models use the default service tier, and the host that served each call is logged. Model ids are exact; an unknown id, or one without tool support, fails the run instead of being fuzzy-matched. Anthropic models get explicit prompt-caching breakpoints (on the system prompt and the newest message), since that API bills every re-sent token otherwise; the other providers cache prefixes automatically. The text the model sees is identical either way; only billing differs. One wire-format adjustment is provider-specific: Alibaba's endpoint rejects a null assistant `content`, so for Qwen it is sent as an empty string, which carries the same meaning.

Seeds, blinding and provenance. Seeds are drawn from a private round key (§5), and the key's commitment is published before the run. All instances are blind. Each archetype gets its seeds once, and its three tiers are built from the same seed, so the tier comparison is paired on one program: *bare* (native aarch64, C, unprotected), *obf* (the same build under MBA, `opaque_const`, control-flow flattening, string encryption, anti-debug and virtualization) and *exotic* (the same logic re-hosted as a bare-metal Cortex-M image, which on a blind instance adds the ISA-identification fact). Every cell records the harness git commit and dirty flag, the image digest, hashes of the system prompt and tool schema, the configuration's path and hash, the round commitment, the serving hosts, the cost and per-call token usage; full transcripts are kept.

Statistics. A cell counts only if it graded cleanly. Samples of one instance (archetype, seed, tier) are averaged first; a model's score is the mean over instances, with a 95% percentile bootstrap interval that resamples instances ($B = 2000$, fixed RNG seed) [6]. A model that completes fewer than 95% of its expected cells is listed but not ranked or banded, since its surviving cells may be the easier ones. Per-tier and per-task-family means are reported next to the composite, because the composite alone hides which skill fails. Models are presented as *score bands*, not ranks: sorted by mean, a model joins the current band while its interval's upper end reaches the lower end of the band leader's interval, and starts a new band otherwise. Within a band the order carries no claim.

Two levels. Table 2 gives both. The full v1 design remains the pre-registered evaluation and is pending funding. Round 0 is identical in everything that can change a score (prompts, tools, budgets, sampling parameters, reasoning effort, routing, roster) except the sample size. v1 pre-registers the same roster as Round 0. Google's Gemini 3.8 Pro is absent because it is not available on OpenRouter, and DeepSeek's pinned V4 Pro snapshot replaces the earlier `deepseek-v3.2`, which no longer has a first-party endpoint. Round 0 runs these 13 models (exact OpenRouter ids):

TABLE 2 · The two evaluation levels. Both are blind, use C targets and the same budgets and routing; only the sample size differs.

	v1 (pre-registered)	Round 0 (preview)
config	`eval/official.toml`	`eval/round0.toml`
archetypes	`protocol`, `keycheck`, `vm`, `dsp`, `transform`, `format`, `malware_surrogate`, `savefile` (8, both levels)	
tiers	bare, obf, exotic	bare, obf, exotic
seeds per archetype	5	1
samples per instance	2	1
episodes per model	240	24
API budget	not yet set	$385
status	pending funding	preliminary preview

`anthropic/claude-opus-5.5`, `anthropic/claude-sonnet-5.5`, `anthropic/claude-haiku-5.5`, `openai/gpt-6.1-sol`, `openai/gpt-6-astra`, `google/gemini-3.8-flash`, `x-ai/grok-4.7`, `mistralai/mistral-large-4-0`, `z-ai/glm-5.3`, `deepseek/deepseek-v4-pro-0813`, `qwen/qwen3.8-max-prime`, `moonshotai/kimi-k3` and `meta/muse-spark-1.3`.

6.2 Round 0: reporting plan

Round 0 gives each model 24 episodes: eight per tier and three per archetype. That is enough to see how capability changes under protection when pooled over models, and to separate models into coarse bands; it is not enough for a ranking, and per-archetype numbers for a single model (three episodes) are too noisy to report. Its results are therefore reported as (i) the headline, capability under protection (*bare→obf→exotic*), as the per-tier mean composite pooled over all banded models, with a 95% percentile bootstrap interval that resamples instances ($B = 2000$); (ii) the models as score bands on a dot plot with 95% intervals, at two decimals; and (iii) per-archetype numbers only pooled over models. Round 0 uses commit–reveal like any official round: the commitment below was published before the run, and the round key is revealed after it.

round commitment `9251359827da55cab55c4c66bed9fff0d6be8c1b331a55be87c155eb88d4c9dd`

These results are published separately and are not part of this paper.

7 Related work

Retrograde inherits the decompilation metric lineage [3] and pushes it to behavioral equivalence under adversarial test. Against SRE-Bench [5]—static, hand-built, expensive, contaminating— Retrograde is generated, behaviorally graded, provenance- and calibration-aware, and self-renewing. Against contaminated question banks and CTF suites (BinMetric [1], Cybench [2]) it is contamination-proof by construction. Relative to SWE-bench [7], whose power is the ecological validity of real repositories and real test suites, Retrograde trades some realism for contamination resistance, behavioral and causal grading, and infinite fresh supply; the two measure different skills (bug-fixing vs. reverse engineering) and are complementary.

8 Limitations

No model results in this paper. This paper describes the approach. The full pre-registered v1 evaluation (240 episodes per model) is pending funding; Round 0 (§6.2) gives each model only 24 episodes, one seed per archetype and tier and one sample, so its per-model intervals are wide, its bands are coarse, and per-archetype per-model numbers are not reported at all. It supports statements about capability under protection pooled over models and about clearly separated bands, not a ranking. **Data-integrity incidents, and the guards they added.** An internal pilot, run before the frozen protocol existed, surfaced three harness artifacts that produced fake near-zero scores, each caught on audit. (1) During one batch the API key was absent and the harness silently *abstained* instead of failing, so the runner recorded ≈0 composites that looked like genuine collapses; the pipeline now refuses to record any episode that made no API call. (2) An API call failed mid-episode (provider capacity or rate limits), and the empty forced submission was graded; such an episode is now recorded as an error, never a score, and re-run. (3) A model wrote integers as hexadecimal literals, which is not valid JSON, and its host forwarded the unparseable tool argument as an empty string, so a forced final submission arrived empty although the answers were present; the harness now parses near-JSON leniently, takes an empty submission once as plain text, and keeps the prompt well inside the context window. All affected pilot episodes were quarantined. Each artifact had first been misread as a model property, which is why the protocol of §6.1 treats every infrastructure failure as an error to re-run rather than as data. Tighter error bars and the human-expert calibration remain to be done. The calibration apparatus is built and validated on the non-LLM panel across seven archetypes and four seeds (28 instances): the composite ranks the four panel agents in their construction-true ability order (oracle 0.999 > dynamic-analysis 0.62 > static-recovery 0.00 > guesser −0.88) at Spearman $\rho = 1.0$, so the score tracks ability on known-ability solvers before any expert is recruited. **External validity (synthetic vs. real targets).** This is the central open question. Every instance is procedurally generated from a compact Behavior IR, not harvested from real software, so the corpus does not reproduce the scale, library bulk, template/STL sprawl, or idiom distribution of a real stripped binary, a commercial firmware image, or a malware sample. The design intent is not photorealism but *isolation*: each archetype models a recognisable real reverse-engineering scenario (a framed wire protocol with a CRC and a guarded state machine, a bytecode interpreter, a license-key check, a firmware signal path, an encrypted boot image, a bank-switched or MMU-translated memory map) so that a specific skill can be measured under contamination-proof, executable ground truth—something no corpus of real binaries can offer, because real binaries leak into training data and cannot be regenerated or behaviorally re-graded at will. Whether competence on these synthetic-but-grounded instances *transfers* to real-world RE is exactly what the frontier runs plus human-expert calibration must establish; until then external validity is unproven, and we do not claim it. The mitigation is a small held-out set of real programs (public-domain implementations of XTEA, CRC-32, FNV-1a and Base64) graded by the same behavioral verifiers, to measure the synthetic-to-real gap directly rather than assume it away; the instances, grader and runner are built and validated on the non-LLM panel, but no frontier-model numbers exist yet. **Scale and logic depth.** A `size` knob scales programs from ~700 to ~3.8K instructions bare (~5.3K on firmware), and to ~22K under the full protection stack. Most of that last step is obfuscation rather than program logic, and SRE-Bench's ~16.9K per program counts lines of source, not instructions, so we do not claim comparable scale.

Each instance is one small workload: an obscure ISA or memory map raises the cost of tooling (disassembly, re-hosting) more than the cost of understanding, and a program of a few thousand instructions, once re-hosted, is cheap to probe by brute force. Deeper archetype logic is future work. **Unrelated code.** Real binaries often consist mostly of library code, and whether agents degrade when the program sits among unrelated code is untested. The planned runtime layer links a pinned libc and RTOS into some instances as a knob (same program, library on or off), with the address ranges of application code as a recovery fact taken from the link map. An agent pays tokens for what it reads, so this ablation will run on a small isolated slice. **Build provenance as facts.** The generator chooses the compiler, optimization level, protections and library versions, so each is known by construction. Blind re-hosted instances already grade the ISA, and mixed-compiler targets grade which compiler produced each function (§3), where the two compilers' traces can mislead; grading the optimization level, protections and library versions on ordinary instances is planned, not implemented. **Firmware peripherals.** Four bare-metal ISAs with a modeled UART are re-hosted, plus seeded synchronous peripherals (bank-switched windows, a custom MMU, an aux-core mailbox, and a programmable security coprocessor), each exercised by the self-consistency gate. *Asynchronous* interrupt handlers (SysTick, DMA) remain a documented extension: nondeterministic, cycle-scheduled timing cannot be reconciled with the deterministic `oracle` without extending the IR to model time, so they are deliberately out of scope for now. **Toolchain pinning.** Native targets compile inside the sandbox image, whose base is pinned to a content digest and whose byte-determining packages (clang 18, gcc 13, binutils, libc) are version-pinned; each instance's manifest records the image digest that built it. Bare-metal targets still compile with the host's cross toolchain (the in-repo clang configurations, or gcc where installed), so the Round 0 *exotic* tier depends on it, and a single pinned build image for every target remains to be done. **Seed exposure.** Non-blind instances list the seed and archetype in agent-visible metadata, and the harness then names the archetype in its prompt. Whoever holds the generator can rebuild every answer from the seed. The generator is not published and the sandbox has no network or repository, but the archetype name is still a hint. Blind instances remove both and carry only HMAC-keyed identifiers drawn from a 2^{62} seed space, so the seed cannot be recovered from what the agent sees; Round 0, v1, the frozen slice and live rounds use them. **Platform settings not yet measured.** The memory-map, string-representation and number-format settings and the license-key workload are verified to keep instances gradeable (§6), but what each one costs a frontier agent is not yet measured. The non-LLM dynamic-analysis agent runs the target on a board that already knows the memory map, so its score does not reflect the memory-map or string settings. **IRT panel size.** Difficulty estimates are only as good as the panel; a larger, more diverse panel sharpens them.

9 Safety and responsible release

The malware tier is a **benign behavioral surrogate**: it shares the control/data-flow shape of a C2/persistence pattern (beacon loop, multi-step arming sequence, dormant→armed→active state machine) with *no payload, no network callback, and nothing weaponizable*. "Activation" emits a token and nothing else happens. No task asks for a working exploit; vulnerability tasks, when added, are localization and understanding only. All target execution is network-isolated and resource-bounded; the agent harness runs the model's tools in a container with no network and no capabilities, and the shipped sandbox view physically ex-

cludes the ground truth. **Release.** No source code is published: the generator, protections, harness, grader and agents stay private, which also keeps live rounds unreproducible by outsiders. What is released is content (this paper, figures, per-cell results with provenance, and transcripts) under CC BY 4.0, plus, if approved, the frozen static slice as binaries and public tasks only. Commit–reveal lets anyone verify that a round's seeds were fixed in advance. The live leaderboard uses fresh seeds, and every artifact and binary carries a keyed canary for leak detection. We did not pre-notify model developers: the benchmark measures public models through their public APIs.

Acknowledgements

Design, typesetting, and editorial production of this preprint are by The Flock Capital Research Department, whose error-diffusion pipeline also printed the plates—both decoded from the benchmark's own objects: the frontispiece from a synthesized call graph, the closing ridge from an instruction bitstream.

References

[1] Anonymous. "BinMetric: A Benchmark for Binary Analysis with Large Language Models". In: *IJCAI*. 2025.

[2] Anonymous. *Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risks of Language Models*. arXiv preprint. 2024.

[3] Anonymous. *LLM4Decompile: Decompiling Binary Code with Large Language Models*. arXiv preprint. 2024.

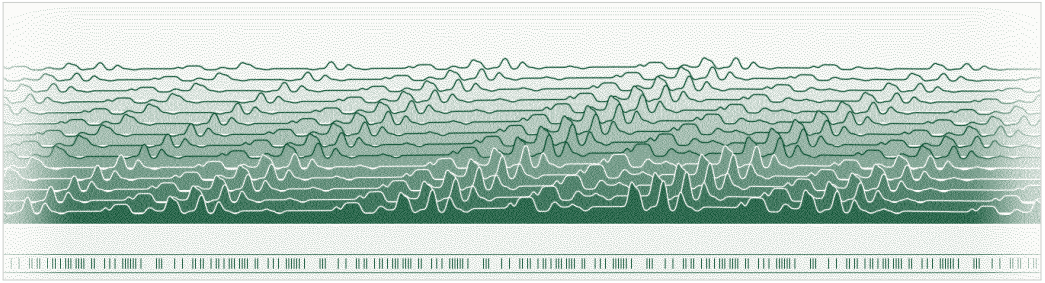
[4] Glenn W. Brier. *Verification of Forecasts Expressed in Terms of Probability*. Monthly Weather Review. 1950.

[5] Columbia DAPLab. "SRE-Bench: A Static Reverse-Engineering Benchmark for Language-Model Agents". In: *arXiv preprint arXiv:2608.11469* (2026).

[6] Bradley Efron. "Bootstrap Methods: Another Look at the Jackknife". In: *The Annals of Statistics* 7.1 (1979), pp. 1–26.

[7] Carlos E. Jimenez et al. *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* ICLR. 2024.

[8] Georg Rasch. "Probabilistic Models for Some Intelligence and Attainment Tests". In: *Danmarks Paedagogiske Institut* (1960).



TRACE · SEED 0X7ACE · THE RIDGE IS DECODED FROM THE BITSTREAM BELOW